



Лекція 21

Безпека web-застосунків. Spring Security

Про Security

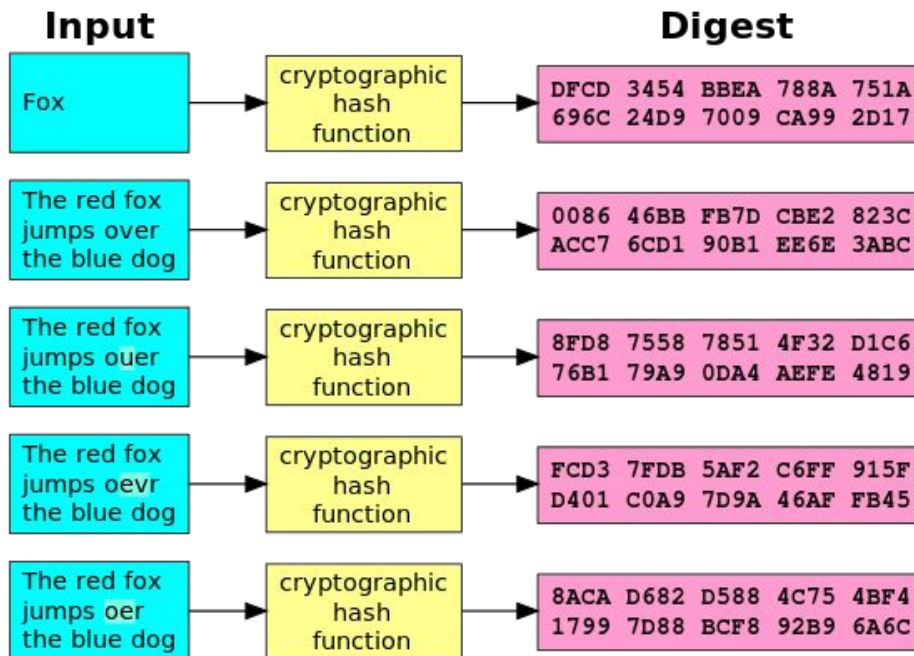
Механізми безпеки - це нефункціональне, додаткове ускладнення ПЗ

Кожно разу додаючи такий механізм ставте собі питання

Яку проблему ми цим вирішуємо?

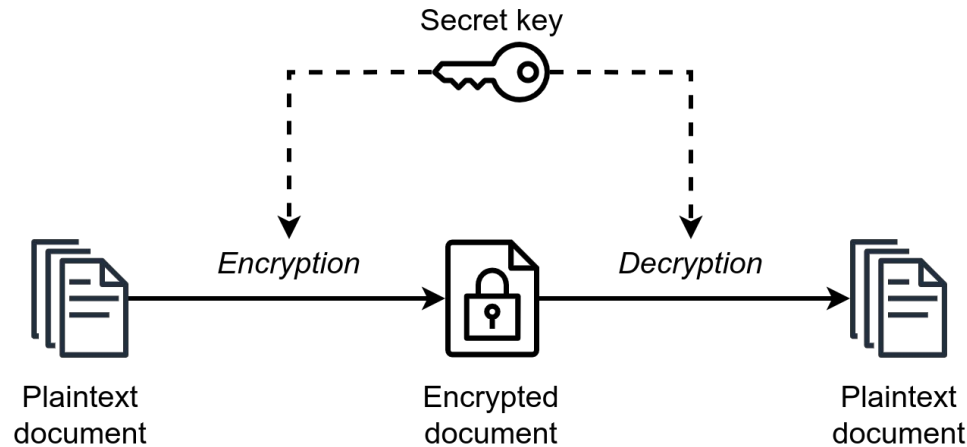
- Щоб зловмисник не отримав доступ до персональної/фінансової та іншої чутливої інформації
- Щоб той, кому це не дозволено, не міг змінити налаштування/статуси/паролі і т.і.
- Щоб систему не можна було “положити”, передавши в неї певні дані
- І т.і.

Hash-функції



MD5
SHA-1
SHA-2
SHA-3
bcrypt
etc.

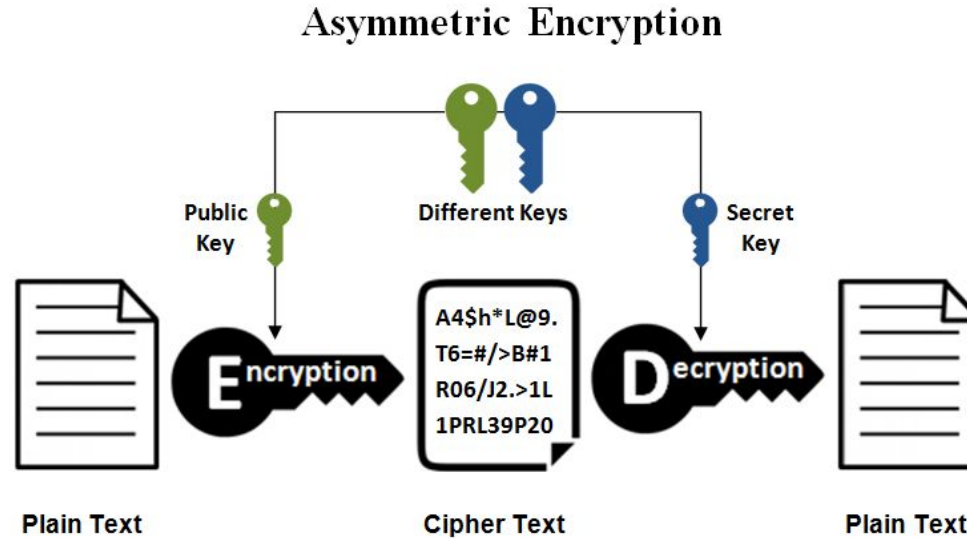
Symmetric-keys cryptography



AES
Blowfish
DES
etc.

https://en.wikipedia.org/wiki/Symmetric-key_algorithm

Asymmetric-keys

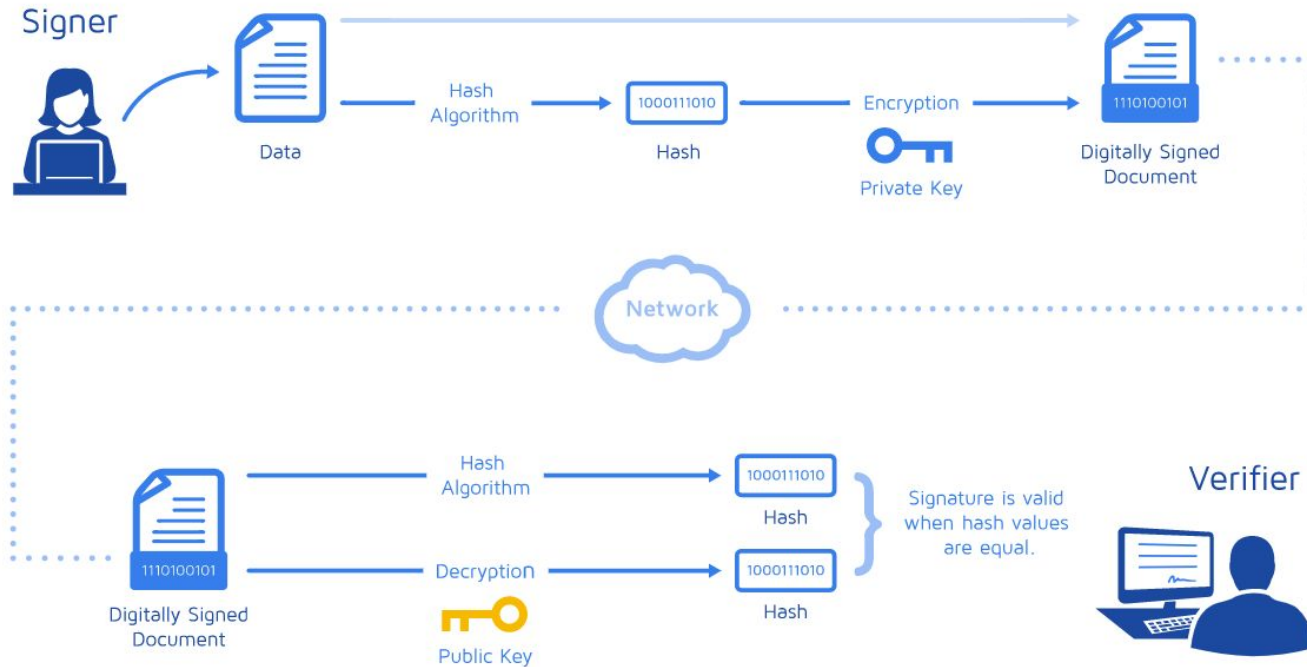


**RSA
etc.**

A message that is encrypted using a public key can only be decrypted using a private key, while also, a message encrypted using a private key can be decrypted using a public key.

<https://www.ssl2buy.com/wiki/symmetric-vs-asymmetric-encryption-what-are-differences>

Digital Signature



Сертифікат відкритого ключа

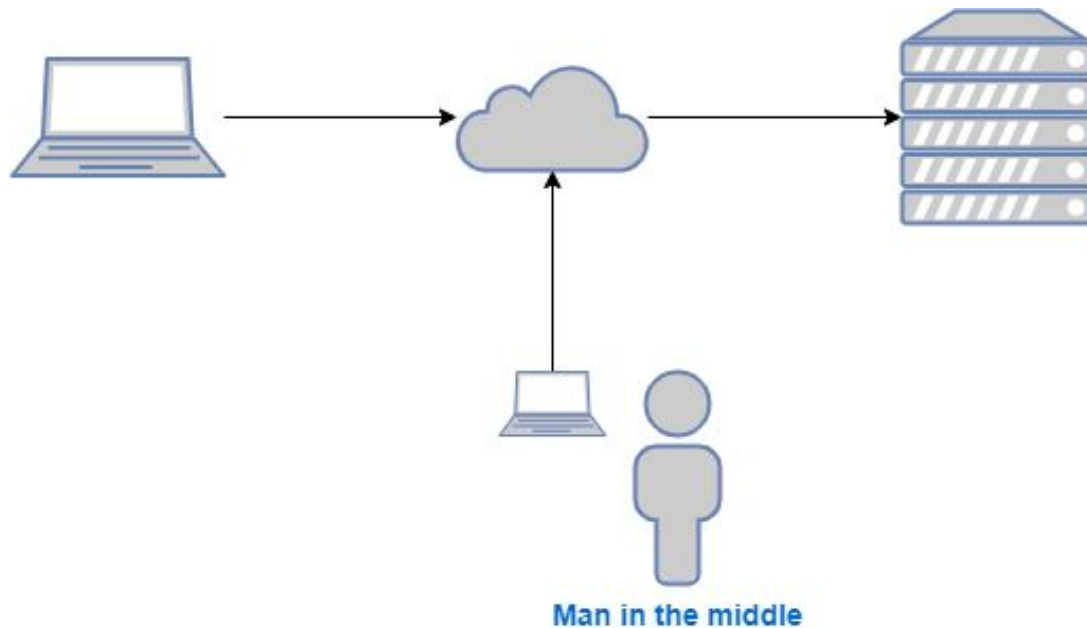
Сертифікат відкритого ключа (англ. Public Key Certificate) - це електронний документ, який містить **публічний ключ** та інформацію про його **власника**, а також

- інформацію про **видавця сертифіката**
- дату видачі та **термін дії** сертифіката
- серійний номер сертифіката
- інші технічні дані

Структура сертифіката відкритого ключа визначена стандартом **X.509**.

Зазвичай, сертифікати випускаються **довіреними центрами сертифікації** (Certification Authority), які перевіряють інформацію про власника ключа та видавця сертифіката перед його видачею.

Захист від перехвату трафіку



HTTPS

1. Перевірка сертифікату серверу
2. Шифрування трафіку (неможливо прочитати)
3. Захист від модифікації запиту/відповіді

Автентифікація

Автентифікація - це процес підтвердження ідентичності користувача або програми перед наданням доступу до обмежених ресурсів або послуг.

Методи автентифікації

- Ім'я + секрет (логін + пароль)
- Ключ + сертифікат
- Біометричні дані (відбиток пальця, тощо)
- SMS, Email, моб. додаток
- Карточка доступу
- Токен
- І т.і.

Вимоги до паролів

- Не менше певної довжини
- Букви + цифри [+ символи]
- Відсутність в словниках паролів
- Обмежена кількість невдалих спроб вводу пароля підряд (обліковий запис блокується)
- Обмеження на використання попередніх паролів під час зміни пароля

На сервері в БД паролі не зберігаються, лише їх геші

Двофакторна автентифікація (2FA)

Двофакторна автентифікація - це метод захисту облікового запису, який вимагає від користувачів підтвердження своєї особи не тільки з використанням пароля, але й з використанням додаткового фактора, такого як мобільний телефон або ключ безпеки.

те що знаю (пароль) + **те що маю** (моб. телефон)

Авторизація

Авторизація - це процес надання дозволу користувачеві на доступ до конкретних ресурсів або функцій системи.

Приклади:

- Використання web-інтерфейсу системи
- Пошук і реєстрація клієнтів
- Перегляд документів
- Доступ до фінансової інформації

Механізми авторизації

На основі Ролей

Кнопку “Створити користувача” показуємо якщо у користувача є роль Адміністратор

На основі Привілей або Прав доступу (privileges or permissions)

Кнопку “Створити користувача” показуємо якщо у користувача є привілея “Редагування користувачів”

Ці механізми можна скомбінувати

- Авторизацію робити на основі Привілей
- Привілеї згрупувати в Ролі, і користувачеві назначати Ролі

Spring Boot Security Autoconfiguration

<dependency>

 <groupId>org.springframework.boot</groupId>

 <artifactId>spring-boot-starter-security</artifactId>

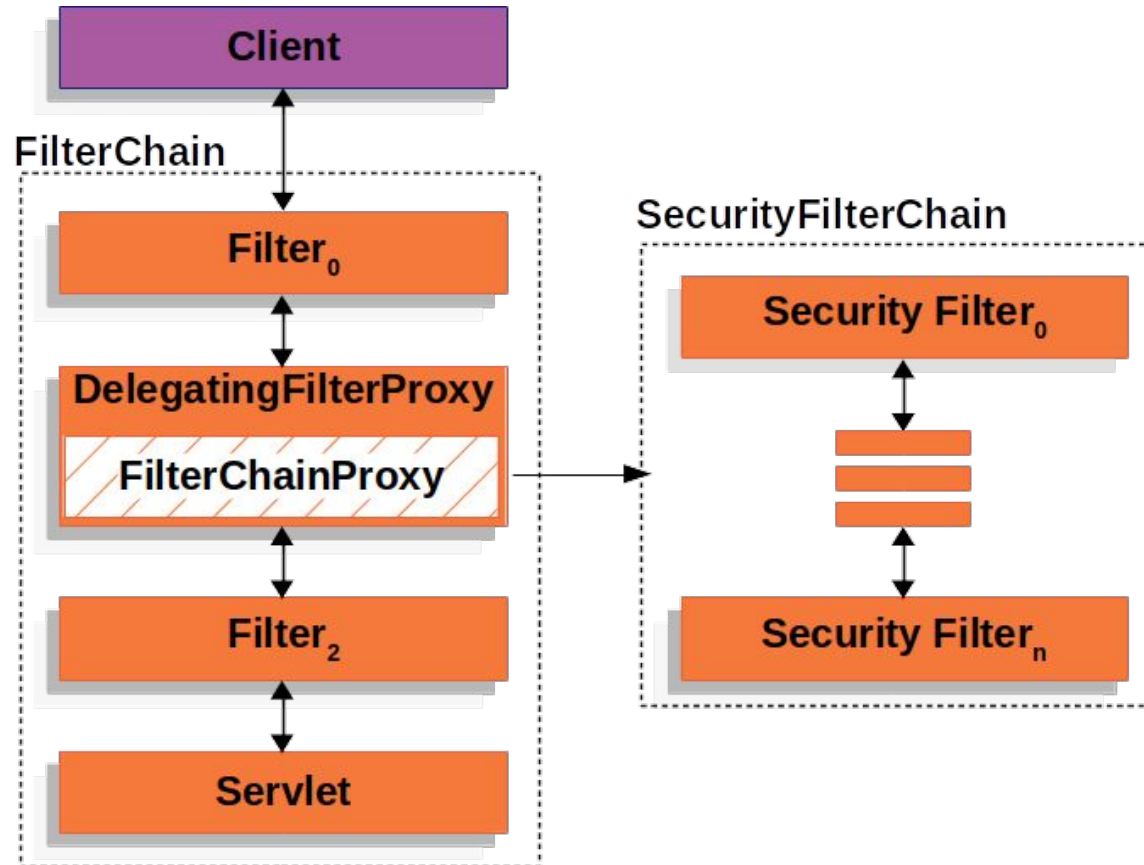
</dependency>

Spring Boot automatically:

- Registers the Filter with a bean named **springSecurityFilterChain** with the Servlet container for every request .
- Creates a **UserDetailsService** bean with a username of *user* and a **randomly generated password** that is logged to the console.
- Require an authenticated user for any interaction with the application
- Generate a default **login form** for you
- Lets the user log out
- Etc.

<https://docs.spring.io/spring-security/reference/servlet/getting-started.html>

Spring Security Architecture



Configure SecurityFilterChain

@Configuration

```
public class SecurityConfig {
```

@Bean

```
public SecurityFilterChain filterChain(HttpSecurity http)
    throws Exception {

    return http
        .authorizeRequests()
            .anyRequest().authenticated()
            .and()
        .formLogin(withDefaults())
        .build();
}
}
```

UserDetailsService

@Service

```
public class UserService implements UserDetailsService {
```

@Override

```
public UserDetails loadUserByUsername(String username)  
    throws UsernameNotFoundException {
```

```
    return
```

```
        userRepository.findByUsername(username)
```

```
        .map(this::toUserDetails)
```

```
        .orElseThrow(() ->
```

```
            new UsernameNotFoundException(
```

```
                "User with name '%s' not found"
```

```
                .formatted(username))
```

```
    );
```

```
}
```

DelegatingPasswordEncoder + bcrypt

```
PasswordEncoder passwordEncoder = PasswordEncoderFactories
```

```
.createDelegatingPasswordEncoder();
```

```
userRepository.save(UserData.builder()
```

```
.username("user")
```

```
.password("{noop}usersecret")
```

```
.role("USER")
```

```
.enabled(true)
```

```
.build()
```

```
);
```

```
userRepository.save(UserData.builder()
```

```
.username("admin")
```

```
// {bcrypt}$2a$10$H7E03AAVh9arrv7kXUJrnevA00rxP7EoYFpW8vXAkLfznnJvb5X0e
```

```
.password(passwordEncoder.encode("adminsecret"))
```

```
.role("ADMIN")
```

```
.enabled(true)
```

```
.build()
```

```
);
```

<https://en.wikipedia.org/wiki/Bcrypt>

```
$2a$12$R9h/cIPz0gi.URNNX3kh20PST9/PgBkqquzi.Ss7KIUg02t0jWMUW
```

Alg	Cost	Salt	Hash
\$2a	12	\$R9h/cIPz0gi.URNNX3kh20PST9/PgBkqquzi.Ss7KIUg02t0jWMUW	

Своя сторінка входу (конфігурація)

@Bean

```
public SecurityFilterChain filterChain(HttpSecurity http)
    throws Exception {

    return http
        .authorizeRequests()
            .requestMatchers("/*.jpg").permitAll()
            .anyRequest().authenticated()
            .and()
        .formLogin(form -> form
            .loginPage("/login").permitAll()
        )
        .build();
}
```

Своя сторінка входу (контроллер + Thymeleaf)

@Controller

```
public class LoginController {  
    @GetMapping("/login")  
    public String login() {  
        return "login";  
    }  
}
```

<dependency>

 <groupId>org.springframework.boot</groupId>

 <artifactId>spring-boot-starter-thymeleaf</artifactId>

</dependency>

<dependency>

 <groupId>org.thymeleaf.extras</groupId>

 <artifactId>thymeleaf-extras-springsecurity6</artifactId>

</dependency>

Своя сторінка входу (login.html)

`src/main/resources/templates/login.html`

```
<html xmlns="http://www.w3.org/1999/xhtml"
xmlns:th="https://www.thymeleaf.org">

<head><title>Please Log In</title></head>

<body>



<h1>Please Log In</h1>

<div th:if="${param.error}">
    Invalid username and password.</div>

<div th:if="${param.logout}">
    You have been logged out.</div>

<form th:action="@{/login}" method="post">
    <div><input type="text" name="username" placeholder="Username"/></div>
    <div><input type="password" name="password"
placeholder="Password"/></div>
    <input type="submit" value="Log in" />
</form></body></html>
```

CSRF Attack

CSRF (Cross-Site Request Forgery) - Міжсайтова підробка запиту - тип вебатаки, що призводить до виконання певних дій від імені користувача на вебсторінці, де останній аутентифікований.

```
public SecurityFilterChain filterChain(  
    HttpSecurity http) throws Exception {  
    return http  
        // ...  
        .csrf() // включено за замовченням  
        // ...  
}
```

Ролі і Привілеї



@Getter

@Builder

```
public class Role {
```

```
    private String id;
```

```
    private List<String> privileges;
```

```
}
```

UserDetails.authorities

```
private UserDetails toUserDetails(UserData user) {  
    return User.withUsername(user.getUsername())  
        .password(user.getPassword())  
        .authorities(collectAuthorities(user.getRole()))  
        .disabled(!user.isEnabled())  
        .build();  
}  
  
private List<GrantedAuthority> collectAuthorities(String role) {  
    return roleRepository.getRole(role)  
        .map(Role::getPrivileges)  
        .stream().flatMap(Collection::stream)  
        .map(priv -> (GrantedAuthority)  
            new SimpleGrantedAuthority("PRIV_" + priv))  
        .toList();  
}
```

Перевірка прав доступу

```
// Щоб працювали анотації на рівні методів,  
// В конфігурації треба додати @EnableMethodSecurity  
@Configuration  
@EnableMethodSecurity  
public class SecurityConfig {..}
```

```
// В контролерах можемо використовувати @PreAuthorize  
@GetMapping("/users")  
@PreAuthorize("hasAuthority('PRIV_USER_MANAGEMENT')")  
public String users(Model model) {  
    // ...  
}
```

Перевірка прав на рівні securityFilterChain

@Bean

```
SecurityFilterChain filterChain(HttpSecurity http) {  
    return http  
        .authorizeRequests(authRequests -> authRequests  
            .requestMatchers("/users")  
            .hasAuthority("PRIV_USER_MANAGEMENT")  
        )  
        // ..  
}
```

А також в Thymeleaf template

```
<div sec:authorize="hasAuthority('PRIV_USER_MANAGEMENT')">  
    <a th:href="@{/users}">Users List</a> <br />  
</div>
```

Дані користувача в коді

```
SecurityContext securityContext =  
    SecurityContextHolder.getContext();  
Authentication authentication =  
    securityContext.getAuthentication();  
  
// principal - дані про користувача  
Object principal = authentication.getPrincipal();  
if (principal instanceof User user) {  
    log.info("Some code executed for '{}'",  
        user.getUsername());  
}  
  
// authorities - права користувача  
if (authentication.getAuthorities().stream()  
    .anyMatch(auth -> auth.getAuthority()  
        .equals("PRIV_USER_MANAGEMENT"))) {  
    log.info("Privilege exists");  
}
```

Basic Authentication

@Bean

```
public SecurityFilterChain basicFilterChain(HttpSecurity http)
    throws Exception {
    return http
        .authorizeHttpRequests(authRequests -> authRequests
            .requestMatchers("/api/**")
            .hasAuthority("PRIV_API_ACCESS")
            .anyRequest().authenticated()
        )
        .httpBasic(withDefaults())
        .build();
}
```

<https://docs.spring.io/spring-security/reference/servlet/authentication/passwords/basic.html>